

```

<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>学習管理アプリ | STUDY TRACKER</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=Noto+
Sans+JP:wght@400;500;700&display=swap" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <style>
    body { font-family: 'Inter', 'Noto Sans JP', sans-serif; }
    .subject-btn-selected {
      background-color: #4f46e5 !important;
      color: white !important;
      border-color: #4f46e5 !important;
    }
    .target-btn-selected {
      background-color: #10b981 !important;
      color: white !important;
      border-color: #10b981 !important;
    }
    .tabular-nums { font-variant-numeric: tabular-nums; }
    .custom-scrollbar::-webkit-scrollbar { width: 4px; }
    .custom-scrollbar::-webkit-scrollbar-thumb { background: #e2e8f0; border-radius: 10px;
  }
  @keyframes pulse-soft {
    0%, 100% { opacity: 0.05; }
    50% { opacity: 0.15; }
  }
  .animate-pulse-soft {
    animation: pulse-soft 3s cubic-bezier(0.4, 0, 0.6, 1) infinite;
  }
</style>
</head>
<body class="bg-gray-50 text-gray-800">

  <div id="date-setting-modal" class="fixed inset-0 bg-black bg-opacity-50 flex items-center
justify-center z-[70] hidden">
    <div class="bg-white p-6 rounded-2xl shadow-2xl max-w-sm w-full mx-4">
      <h3 class="text-lg font-bold mb-2"><span id="date-setting-target-name"></span>の
日程設定</h3>
      <p class="text-sm text-gray-500 mb-4">試験日を選択してください。</p>
      <input type="date" id="date-target-date-input" class="w-full px-4 py-2 border
border-gray-300 rounded-lg mb-6 outline-none focus:ring-2 focus:ring-emerald-500">

```

```

        <div class="flex gap-2">
            <button id="close-date-modal" class="flex-1 bg-gray-200 text-gray-700 py-2
rounded-lg font-bold">キャンセル</button>
            <button id="save-date-btn" class="flex-1 bg-emerald-600 text-white py-2
rounded-lg font-bold hover:bg-emerald-700">保存する</button>
        </div>
    </div>
</div>

<div id="daily-goal-modal" class="fixed inset-0 bg-black bg-opacity-50 flex items-center
justify-center z-[70] hidden">
    <div class="bg-white p-6 rounded-2xl shadow-2xl max-w-sm w-full mx-4">
        <h3 class="text-lg font-bold mb-2">1日の目標時間の設定</h3>
        <p class="text-sm text-gray-500 mb-4">今日の目標勉強時間を入力してください。</p>
        <div class="relative">
            <input type="number" id="daily-goal-input" class="w-full px-4 py-3 border
border-gray-300 rounded-lg mb-6 outline-none focus:ring-2 focus:ring-indigo-500 text-xl
font-bold tabular-nums pr-12" placeholder="3">
            <span class="absolute right-4 top-3.5 text-gray-400 font-bold">時間</span>
        </div>
        <div class="flex gap-2">
            <button id="close-goal-modal" class="flex-1 bg-gray-200 text-gray-700 py-2
rounded-lg font-bold">キャンセル</button>
            <button id="save-goal-btn" class="flex-1 bg-indigo-600 text-white py-2 rounded-lg
font-bold hover:bg-indigo-700">保存する</button>
        </div>
    </div>
</div>

<div id="app" class="max-w-4xl mx-auto p-4 md:p-8">
    <div id="loading-screen" class="fixed inset-0 bg-white bg-opacity-75 flex items-center
justify-center z-50">
        <div class="flex flex-col items-center">
            <svg class="animate-spin h-10 w-10 text-indigo-600"
xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
                <circle class="opacity-25" cx="12" cy="12" r="10" stroke="currentColor"
stroke-width="4"></circle>
                <path class="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0
5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3
7.938l3-2.647z"></path>
            </svg>
            <p class="mt-4 text-gray-600">読み込み中...</p>
        </div>
    </div>

    <header class="flex justify-between items-center mb-6">
        <h1 class="text-2xl font-black text-indigo-600 tracking-tight">STUDY
TRACKER</h1>

```

```
<div class="bg-indigo-600 text-white px-4 py-1.5 rounded-full text-sm font-bold shadow-md">
  <span id="points-display">0</span> P
</div>
</header>
```

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
  <div class="md:col-span-2 bg-white p-6 rounded-2xl shadow-sm border-b-4 border-indigo-500">
    <h2 class="text-lg font-bold text-gray-500">マイダッシュボード</h2>
    <div id="countdown-display" class="mt-1 text-sm font-bold text-emerald-600 italic">
```

目標を選択するとカウントダウンが表示されます

```
</div>
</div>
<div class="bg-white p-6 rounded-2xl shadow-sm flex flex-col justify-center items-center border border-gray-100">
  <p class="text-[10px] font-bold text-gray-400 uppercase tracking-widest mb-1">Daily Goal Status</p>
  <div id="daily-percentage" class="text-3xl font-black text-indigo-600">0%</div>
</div>
</div>
```

```
<section class="bg-white p-6 rounded-2xl shadow-sm mb-8 border border-gray-100">
  <div class="flex justify-between items-center mb-4">
    <h3 class="text-lg font-bold flex items-center gap-2 text-gray-700">
       今日の進捗
    </h3>
    <button id="open-goal-modal" class="text-[10px] bg-gray-50 hover:bg-gray-100 text-gray-500 px-2 py-1 rounded-md border border-gray-200 transition-colors">目標設定</button>
  </div>
```

```
</h3>
  <div class="text-right">
    <span id="daily-total-display" class="font-bold text-indigo-600">0時間00分</span>
  </div>
  <span class="text-xs text-gray-400"> / </span>
  <span class="text-xs text-gray-400" id="daily-goal-display">--時間</span>
</div>
</div>
```

```
<div class="w-full bg-gray-100 rounded-full h-3 mb-1">
  <div id="daily-progress-bar" class="bg-indigo-600 h-3 rounded-full transition-all duration-700 ease-out" style="width: 0%"></div>
</div>
</section>
```

```
<div class="grid grid-cols-1 lg:grid-cols-3 gap-6 mb-8">
  <div class="lg:col-span-2 space-y-4">
    <div class="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
      <h3 class="text-sm font-bold text-gray-400 mb-4 flex items-center gap-2">
```

```

        <span class="w-5 h-5 bg-gray-100 rounded-full flex items-center justify-center
text-[10px]">1</span>
        目標の選択
    </h3>
    <div id="target-selection-container" class="grid grid-cols-1 sm:grid-cols-2
gap-3"></div>
</div>

<div class="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
    <h3 class="text-sm font-bold text-gray-400 mb-4 flex items-center gap-2">
        <span class="w-5 h-5 bg-gray-100 rounded-full flex items-center justify-center
text-[10px]">2</span>
        教科の選択
    </h3>
    <div id="subject-selection-container" class="grid grid-cols-3 sm:grid-cols-4
gap-3"></div>

    <div id="sub-subject-area" class="mt-4 p-4 bg-indigo-50/50 rounded-xl border
border-indigo-100 hidden">
        <p class="text-[10px] font-bold text-indigo-400 mb-3 uppercase
tracking-wider" id="sub-subject-title">詳細科目</p>
        <div id="sub-subject-container" class="grid grid-cols-2 sm:grid-cols-3
gap-2"></div>
    </div>
</div>
</div>
</div>

<div class="bg-white p-8 rounded-2xl shadow-lg flex flex-col items-center
justify-center border-2 border-indigo-50 relative overflow-hidden min-h-[360px]">
    <div id="timer-bg-pulse" class="absolute inset-0 bg-indigo-500 opacity-0
transition-all duration-500"></div>
    <div class="relative z-10 text-center w-full flex flex-col items-center justify-center
flex-1">
        <div id="timer-settings" class="flex justify-center gap-2 mb-4 transition-opacity
duration-300">
            <div class="flex items-center gap-1 bg-indigo-50 px-2 py-1 rounded-md
border border-indigo-100">
                <label class="text-[10px] font-bold text-indigo-400">勉強</label>
                <input type="number" id="study-time-input" value="25" min="1" max="180"
class="w-10 bg-transparent text-center font-bold text-indigo-700 outline-none">
                <span class="text-[10px] text-indigo-400">分</span>
            </div>
            <div class="flex items-center gap-1 bg-emerald-50 px-2 py-1 rounded-md
border border-emerald-100">
                <label class="text-[10px] font-bold text-emerald-500">休憩</label>
                <input type="number" id="break-time-input" value="5" min="1" max="60"
class="w-10 bg-transparent text-center font-bold text-emerald-700 outline-none">
                <span class="text-[10px] text-emerald-500">分</span>
            </div>
        </div>
    </div>
</div>

```

```

    </div>
  </div>

  <div id="pomodoro-display" class="mb-1 h-8 flex items-center justify-center">
    <p class="text-[10px] text-gray-400 bg-gray-100 px-4 py-1 rounded-full
inline-block uppercase tracking-widest">Ready to Start</p>
  </div>

  <div id="main-timer-display" class="text-7xl font-black my-2 text-gray-800
tabular-nums tracking-tighter">00:00:00</div>
  <p id="current-config-display" class="text-[10px] font-bold text-gray-400
uppercase tracking-widest mt-4">教科を選んでください</p>

  <div class="relative z-10 w-full space-y-3 mt-4">
    <button id="start-btn" class="w-full bg-indigo-600 text-white font-bold py-4
rounded-xl shadow-lg shadow-indigo-200 hover:bg-indigo-700 disabled:bg-gray-100
disabled:text-gray-300 disabled:shadow-none transition-all" disabled>学習開始</button>
    <div class="grid grid-cols-2 gap-3">
      <button id="pause-btn" class="bg-amber-400 text-white font-bold py-3
rounded-xl hover:bg-amber-500 disabled:bg-gray-50 disabled:text-gray-200" disabled>一時
停止</button>
      <button id="finish-btn" class="bg-rose-500 text-white font-bold py-3
rounded-xl hover:bg-rose-600 disabled:bg-gray-50 disabled:text-gray-200" disabled>完了・保
存</button>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>

<div class="grid grid-cols-1 lg:grid-cols-2 gap-6">
  <div class="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
    <h3 class="text-lg font-bold mb-6 text-gray-700">本日の学習科目</h3>
    <div class="flex flex-col items-center">
      <div class="w-full max-w-[200px] mb-8">
        <canvas id="study-chart"></canvas>
      </div>
      <div id="study-log-summary" class="w-full space-y-2"></div>
    </div>
  </div>
</div>

  <div class="bg-white p-6 rounded-2xl shadow-sm border border-gray-100">
    <h3 class="text-lg font-bold mb-6 text-gray-700">本日の学習履歴</h3>
    <div id="study-log-list" class="space-y-3 max-h-[350px] overflow-y-auto pr-2
custom-scrollbar">
      <p class="text-gray-400 italic text-center py-10">データがありません</p>
    </div>
  </div>
</div>

```

```
</div>
</div>
```

```
<script type="module">
  import { initializeApp } from "https://www.gstatic.com/firebasejs/11.6.1/firebase-app.js";
  import { getAuth, onAuthStateChanged, signInAnonymously } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-auth.js";
  import { getFirestore, doc, setDoc, getDoc, collection, addDoc, query, where,
onSnapshot, serverTimestamp, updateDoc } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-firestore.js";

  // --- Firebaseの設定 ---
  // ※ 実際にサーバーと同期させて永続化する場合は、あなたのFirebaseプロジェクトの
  Configを入れてください。
  // 未設定でもローカルで動くようにモック(localStorage)への切り替えロジックを挟んでいます。
  const firebaseConfig = {
    apiKey: "YOUR_API_KEY",
    authDomain: "YOUR_AUTH_DOMAIN",
    projectId: "YOUR_PROJECT_ID",
    storageBucket: "YOUR_STORAGE_BUCKET",
    messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
    appId: "YOUR_APP_ID"
  };

  let db = null;
  let auth = null;
  let isLocalMode = true;

  try {
    if (firebaseConfig.apiKey !== "YOUR_API_KEY") {
      const app = initializeApp(firebaseConfig);
      auth = getAuth(app);
      db = getFirestore(app);
      isLocalMode = false;
    }
  } catch (e) {
    console.warn("Firebase initialization failed. Running in Local Mode.", e);
  }

  // --- DOM Elements ---
  const loadingScreen = document.getElementById('loading-screen');
  const startBtn = document.getElementById('start-btn');
  const pauseBtn = document.getElementById('pause-btn');
  const finishBtn = document.getElementById('finish-btn');
  const mainTimerDisplay = document.getElementById('main-timer-display');
  const pomodoroDisplay = document.getElementById('pomodoro-display');
  const countdownDisplay = document.getElementById('countdown-display');
```

```

const dateModal = document.getElementById('date-setting-modal');
const goalModal = document.getElementById('daily-goal-modal');
const subSubjectArea = document.getElementById('sub-subject-area');
const subSubjectContainer = document.getElementById('sub-subject-container');
const studyTimeInput = document.getElementById('study-time-input');
const breakTimeInput = document.getElementById('break-time-input');
const timerSettingsDiv = document.getElementById('timer-settings');

// --- Global State ---
let studentChart = null;
let targetDates = JSON.parse(localStorage.getItem('targetDates')) || {};
let dailyGoalHours = parseInt(localStorage.getItem('dailyGoalHours')) || 3;
let currentSubject = null;
let currentTarget = null;
let localLogs = JSON.parse(localStorage.getItem('localLogs')) || [];

// Timer State
let timerRunning = false;
let currentPhase = 'IDLE';
let currentPhaseType = 'STUDY';
let virtualTimelineStart = null;
let pausedTimelineSeconds = 0;
let lastReportedPhaseType = 'STUDY';
let sessionTotalStudySeconds = 0;
let phaseRemainingSeconds = 0;
let timerInterval = null;

const targets = ["中間定期考査", "期末定期考査", "模擬試験", "共通テスト", "英検", "漢検", "その他"];
const subjectsData = { "国語": null, "数学": null, "英語": null, "理科": ["物理", "化学", "生物", "地学"], "社会": ["地理", "日本史", "世界史", "政経", "倫理"], "情報": null, "その他": null };
const subjectColors = {
  "国語": "#f87171", "数学": "#60a5fa", "英語": "#34d399",
  "理科": "#fbbf24", "社会": "#94a3b8", "情報": "#f59e0b", "その他": "#94a3b8",
  "物理": "#fbbf24", "化学": "#f59e0b", "生物": "#d97706", "地学": "#b45309",
  "地理": "#fbbf24", "日本史": "#f97316", "世界史": "#ea580c", "政経": "#c2410c", "倫理": "#9a3412"
};

// --- App Initializer ---
if (!isLocalMode) {
  signInAnonymously(auth).catch(err => console.error("Auth error:", err));
  onAuthStateChanged(auth, async (user) => {
    if (user) {
      const userDocRef = doc(db, 'users', user.uid);
      const userDoc = await getDoc(userDocRef);
      if (!userDoc.exists()) {

```

```

        await setDoc(userDocRef, { points: 0, targetDates: {}, dailyGoalHours: 3,
totalDuration: 0 });
    } else {
        const data = userDoc.data();
        targetDates = data.targetDates || {};
        dailyGoalHours = data.dailyGoalHours || 3;
    }
    setupFirestoreListeners(user);
    initDashboard();
}
});
} else {
    // ローカルモードの初期化
    setTimeout(() => {
        initDashboard();
        updateStatsUI(getLocalTodayLogs(), localLogs);
    }, 500);
}

function initDashboard() {
    document.getElementById('daily-goal-display').textContent = dailyGoalHours;
    renderTargets();
    renderMainSubjects();
    phaseRemainingSeconds = parseInt(studyTimeInput.value) * 60;
    loadingScreen.style.display = 'none';
    updateCountdown();
}

function getLocalTodayLogs() {
    const todayStr = new Date().toDateStr();
    return localLogs.filter(l => new Date(l.startTime).toDateStr() === todayStr);
}

const setupFirestoreListeners = (user) => {
    onSnapshot(doc(db, 'users', user.uid), (snap) => {
        if (snap.exists()) {
            const data = snap.data();
            document.getElementById('points-display').textContent = data.points || 0;
            targetDates = data.targetDates || {};
            dailyGoalHours = data.dailyGoalHours || 3;
            document.getElementById('daily-goal-display').textContent = dailyGoalHours;
            updateCountdown();
        }
    });
    onSnapshot(query(collection(db, 'study_logs'), where("userId", "==", user.uid)),
(snapshot) => {
        const logs = snapshot.docs.map(d => {
            const data = d.data();

```



```

        return { ...data, startTime: data.startTime?.toDate(), endTime:
data.endTime?.toDate() };
    });
    const todayStr = new Date().toDateString();
    const todayLogs = logs.filter(l => l.startTime && l.startTime.toDateString() ===
todayStr);
    updateStatsUI(todayLogs, logs);
  });
};

// --- Date Config Modal ---
let activeTargetForDate = "";
function openDateModal(target) {
  activeTargetForDate = target;
  document.getElementById('date-setting-target-name').textContent = target;
  document.getElementById('date-target-date-input').value = targetDates[target] || "";
  dateModal.classList.remove('hidden');
}

document.getElementById('close-date-modal').onclick = () =>
dateModal.classList.add('hidden');

document.getElementById('save-date-btn').onclick = async () => {
  const dateStr = document.getElementById('date-target-date-input').value;
  if (!dateStr) return;
  targetDates[activeTargetForDate] = dateStr;

  if (!isLocalMode && auth.currentUser) {
    await updateDoc(doc(db, 'users', auth.currentUser.uid), { targetDates });
  } else {
    localStorage.setItem('targetDates', JSON.stringify(targetDates));
    updateCountdown();
    renderTargets();
  }
  dateModal.classList.add('hidden');
};

// --- Goal Modal ---
document.getElementById('open-goal-modal').onclick = () => {
  document.getElementById('daily-goal-input').value = dailyGoalHours;
  goalModal.classList.remove('hidden');
};
document.getElementById('close-goal-modal').onclick = () =>
goalModal.classList.add('hidden');
document.getElementById('save-goal-btn').onclick = async () => {
  const val = parseInt(document.getElementById('daily-goal-input').value);
  if (isNaN(val) || val <= 0) return;
  dailyGoalHours = val;

```

```

    if (!isLocalMode && auth.currentUser) {
      await updateDoc(doc(db, 'users', auth.currentUser.uid), { dailyGoalHours: val });
    } else {
      localStorage.setItem('dailyGoalHours', val);
      document.getElementById('daily-goal-display').textContent = dailyGoalHours;
      updateStatsUI(getLocalTodayLogs(), localLogs);
    }
    goalModal.classList.add('hidden');
  };

  // --- Timer Logic ---
  studyTimeInput.addEventListener('change', () => {
    if (currentPhase === 'IDLE') phaseRemainingSeconds =
      parseInt(studyTimeInput.value) * 60;
  });

  startBtn.onclick = () => {
    if (!currentSubject || !currentTarget) return;
    if (currentPhase === 'IDLE') {
      if ("Notification" in window && Notification.permission === "default")
        Notification.requestPermission();
      currentPhaseType = 'STUDY';
      pausedTimelineSeconds = 0;
      lastReportedPhaseType = 'STUDY';
      sessionTotalStudySeconds = 0;
    }
    timerRunning = true;
    currentPhase = 'RUNNING';
    virtualTimelineStart = Date.now() - (pausedTimelineSeconds * 1000);

    timerSettingsDiv.style.opacity = "0.3";
    studyTimeInput.disabled = true;
    breakTimeInput.disabled = true;
    startBtn.disabled = true;
    pauseBtn.disabled = false;
    finishBtn.disabled = false;

    timerTick();
    timerInterval = setInterval(timerTick, 1000);
  };

  function timerTick() {
    if (!timerRunning) return;
    const studyPeriod = parseInt(studyTimeInput.value) * 60;
    const breakPeriod = parseInt(breakTimeInput.value) * 60;
    const cycleSeconds = studyPeriod + breakPeriod;
    if (cycleSeconds <= 0) return;
  }

```

```

const elapsedTimelineSeconds = Math.floor((Date.now() - virtualTimelineStart) /
1000);
const cycles = Math.floor(elapsedTimelineSeconds / cycleSeconds);
const remainder = elapsedTimelineSeconds % cycleSeconds;

let calculatedPhaseType = 'STUDY';
let remSec = 0;
let currentStudySec = 0;

if (remainder < studyPeriod) {
  calculatedPhaseType = 'STUDY';
  remSec = studyPeriod - remainder;
  currentStudySec = cycles * studyPeriod + remainder;
} else {
  calculatedPhaseType = 'BREAK';
  remSec = cycleSeconds - remainder;
  currentStudySec = (cycles + 1) * studyPeriod;
}

if (calculatedPhaseType !== lastReportedPhaseType) {
  playNotificationSound();
  if (calculatedPhaseType === 'BREAK') {
    notifyUser("学术管理", "勉強時間終了！少し休憩しましょう！");
  } else {
    notifyUser("学术管理", "休憩終了！勉強を再開しましょう！");
  }
  lastReportedPhaseType = calculatedPhaseType;
}

sessionTotalStudySeconds = currentStudySec;
phaseRemainingSeconds = remSec;

renderMainTimer(sessionTotalStudySeconds);
renderSubTimer(calculatedPhaseType, phaseRemainingSeconds);
updateUIForPhase(calculatedPhaseType);
}

pauseBtn.onclick = () => {
  if (!timerRunning) return;
  clearInterval(timerInterval);
  timerRunning = false;
  currentPhase = 'PAUSED';
  pausedTimelineSeconds = Math.floor((Date.now() - virtualTimelineStart) / 1000);
  pomodoroDisplay.innerHTML = `
```

```

startBtn.disabled = false;
pauseBtn.disabled = true;
startBtn.textContent = "再開";
};

finishBtn.onclick = async () => {
  if (timerRunning) clearInterval(timerInterval);
  pausedTimelineSeconds = Math.floor((Date.now() - virtualTimelineStart) / 1000);

  const studyPeriod = parseInt(studyTimeInput.value) * 60;
  const cycleSeconds = studyPeriod + (parseInt(breakTimeInput.value) * 60);

  let finalStudySeconds = 0;
  if (cycleSeconds > 0) {
    const cycles = Math.floor(pausedTimelineSeconds / cycleSeconds);
    const remainder = pausedTimelineSeconds % cycleSeconds;
    finalStudySeconds = remainder < studyPeriod ? (cycles * studyPeriod +
remainder) : ((cycles + 1) * studyPeriod);
  }

  if (finalStudySeconds >= 1) {
    const logData = {
      subject: currentSubject,
      target: currentTarget,
      duration: finalStudySeconds,
      startTime: new Date(Date.now() - (pausedTimelineSeconds * 1000)),
      endTime: new Date()
    };

    if (!isLocalMode && auth.currentUser) {
      logData.userId = auth.currentUser.uid;
      await addDoc(collection(db, 'study_logs'), logData);
      const userRef = doc(db, 'users', auth.currentUser.uid);
      const snap = await getDoc(userRef);
      const newTotal = (snap.data()?.totalDuration || 0) + finalStudySeconds;
      await updateDoc(userRef, { totalDuration: newTotal, points: Math.floor(newTotal
/ 60) });
    } else {
      localLogs.push(logData);
      localStorage.setItem('localLogs', JSON.stringify(localLogs));
      const currentPoints = parseInt(localStorage.getItem('localPoints')) || 0;
      localStorage.setItem('localPoints', currentPoints + Math.floor(finalStudySeconds
/ 60));

      document.getElementById('points-display').textContent =
localStorage.getItem('localPoints');
      updateStatsUI(getLocalTodayLogs(), localLogs);
    }
  }
}

```

```

    resetTimer();
};

function resetTimer() {
    currentPhase = 'IDLE';
    currentPhaseType = 'STUDY';
    timerRunning = false;
    clearInterval(timerInterval);
    pausedTimelineSeconds = 0;
    sessionTotalStudySeconds = 0;
    lastReportedPhaseType = 'STUDY';

    timerSettingsDiv.style.opacity = "1";
    studyTimeInput.disabled = false;
    breakTimeInput.disabled = false;
    phaseRemainingSeconds = parseInt(studyTimeInput.value) * 60;

    renderMainTimer(0);
    pomodoroDisplay.innerHTML = `
```

```

    osc.frequency.setValueAtTime(880, ctx.currentTime);
    osc.frequency.exponentialRampToValueAtTime(1100, ctx.currentTime + 0.1);
    gain.gain.setValueAtTime(0.3, ctx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.5);
    osc.connect(gain); gain.connect(ctx.destination);
    osc.start(); osc.stop(ctx.currentTime + 0.5);
  } catch (e) { console.log("Audio play failed"); }
}

// --- UI Rendering ---
function renderMainTimer(totalSec) {
  const h = Math.floor(totalSec / 3600).toString().padStart(2, '0');
  const m = Math.floor((totalSec % 3600) / 60).toString().padStart(2, '0');
  const s = (totalSec % 60).toString().padStart(2, '0');
  mainTimerDisplay.textContent = `${h}:${m}:${s}`;
}

function renderSubTimer(phaseType, remSec) {
  const m = Math.floor(remSec / 60).toString().padStart(2, '0');
  const s = (remSec % 60).toString().padStart(2, '0');
  if (phaseType === 'STUDY') {
    pomodoroDisplay.innerHTML = `
```

```

    b.onclick = () => {
      if (timerRunning || currentPhase === 'PAUSED') return;
      document.querySelectorAll('#target-selection-container button').forEach(el =>
el.classList.remove('target-btn-selected'));
      b.classList.add('target-btn-selected');
      currentTarget = t;
      updateCountdown(); checkReady();
    };

    const edit = document.createElement('button');
    edit.className = "p-3 bg-white border border-gray-100 rounded-xl
hover:bg-emerald-50 text-emerald-600 transition-colors";
    edit.innerHTML = `<svg class="w-3.5 h-3.5" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 02-2V7a2 2 0 00-2-2H5a2 2 0 00-2 2v12a2 2
0 002 2z"></path></svg>`;
    edit.onclick = (e) => { e.stopPropagation(); openDateModal(t); };

    wrap.appendChild(b); wrap.appendChild(edit); container.appendChild(wrap);
  });
}

function renderMainSubjects() {
  const container = document.getElementById('subject-selection-container');
  container.innerHTML = '';
  Object.keys(subjectsData).forEach(s => {
    const b = document.createElement('button');
    b.className = "p-2 border border-gray-100 rounded-xl text-xs font-bold
transition-all text-gray-400 bg-gray-50/30 hover:border-indigo-300";
    b.textContent = s;
    b.onclick = () => {
      if (timerRunning || currentPhase === 'PAUSED') return;
      document.querySelectorAll('#subject-selection-container button').forEach(el =>
el.classList.remove('subject-btn-selected'));
      b.classList.add('subject-btn-selected');

      if (subjectsData[s]) {
        renderSubSubjects(s, subjectsData[s]);
        currentSubject = null; startBtn.disabled = true;
        document.getElementById('current-config-display').textContent = "詳細科目を
選択してください";
      } else {
        subSubjectArea.classList.add('hidden');
        currentSubject = s; checkReady();
      }
    };
    container.appendChild(b);
  });
}

```

```

}

function renderSubSubjects(parent, list) {
  subSubjectArea.classList.remove('hidden');
  document.getElementById('sub-subject-title').textContent = `${parent} 詳細`;
  subSubjectContainer.innerHTML = "";
  list.forEach(sub => {
    const b = document.createElement('button');
    b.className = "p-2 border border-indigo-100 rounded-lg text-[10px] font-bold
transition-all text-indigo-400 bg-white shadow-sm hover:bg-indigo-50";
    b.textContent = sub;
    b.onclick = () => {
      if (timerRunning || currentPhase === 'PAUSED') return;
      document.querySelectorAll('#sub-subject-container button').forEach(el =>
el.classList.remove('subject-btn-selected'));
      b.classList.add('subject-btn-selected');
      currentSubject = sub; checkReady();
    };
    subSubjectContainer.appendChild(b);
  });
}

function checkReady() {
  const display = document.getElementById('current-config-display');
  if (currentTarget && currentSubject) {
    display.textContent = `${currentTarget} > ${currentSubject}`;
    display.className = "text-[10px] font-bold text-white bg-indigo-500 py-1.5 px-4
rounded-full inline-block shadow-md uppercase tracking-wider";
    if (currentPhase === 'IDLE' || currentPhase === 'PAUSED') startBtn.disabled =
false;
  } else {
    display.textContent = currentTarget ? `${currentTarget} > 教科未選択` : "教科を選
んでください";
    display.className = "text-[10px] font-bold text-gray-400 uppercase
tracking-widest mt-4";
  }
}

function updateCountdown() {
  if (!currentTarget) {
    countdownDisplay.textContent = "目標を選択してください"; return;
  }
  const dateStr = targetDates[currentTarget];
  if (!dateStr) {
    countdownDisplay.textContent = `${currentTarget}の日程が未設定です`; return;
  }
  const targetDate = new Date(dateStr); targetDate.setHours(0,0,0,0);
  const today = new Date(); today.setHours(0,0,0,0);

```



```

const diff = Math.ceil((targetDate - today) / (1000 * 60 * 60 * 24));

if (diff === 0) {
  countdownDisplay.textContent = "🎉 本日当日です！最高の準備を！";
} else if (diff > 0) {
  countdownDisplay.innerHTML = `${currentTarget}まで あと <span class="text-base text-emerald-600">${diff}</span> 日`;
} else {
  countdownDisplay.textContent = "試験は終了しました。次へ進もう！";
}
}

function updateStatsUI(todayLogs, allLogs) {
  const dailyTotalSec = todayLogs.reduce((sum, l) => sum + l.duration, 0);
  document.getElementById('daily-total-display').textContent =
`${Math.floor(dailyTotalSec / 3600)}時間${Math.floor((dailyTotalSec % 3600) / 60)}分`;

  const percentage = Math.min(Math.floor((dailyTotalSec / (dailyGoalHours * 3600)) *
100), 100);
  document.getElementById('daily-progress-bar').style.width = `${percentage}%`;
  document.getElementById('daily-percentage').textContent = `${percentage}%`;

  const totals = {};
  todayLogs.forEach(l => totals[l.subject] = (totals[l.subject] || 0) + l.duration);

  if (studentChart) studentChart.destroy();
  const labels = Object.keys(totals);
  if (labels.length > 0) {
    studentChart = new
Chart(document.getElementById('study-chart').getContext('2d'), {
      type: 'doughnut',
      data: {
        labels: labels,
        datasets: [{ data: Object.values(totals), backgroundColor: labels.map(k =>
subjectColors[k] || '#e2e8f0'), borderWidth: 0 }]
      },
      options: { cutout: '85%', plugins: { legend: { display: false } } }
    });
  }

  const summary = document.getElementById('study-log-summary');
  summary.innerHTML = labels.length === 0 ? `
```

```

</div>`).join("");

const logList = document.getElementById('study-log-list');
logList.innerHTML = todayLogs.length === 0 ? `<p class="text-gray-400 italic
text-center py-10">データがありません</p>` : [...todayLogs].reverse().map(l => `
  <div class="p-3 bg-white border border-gray-100 rounded-xl shadow-sm mb-2">
    <div class="flex justify-between items-center mb-1">
      <span class="px-1.5 py-0.5 bg-indigo-50 text-indigo-600 rounded text-[9px]
font-bold">${l.target}</span>
      <span class="text-[9px] text-gray-400 font-mono">${new
Date(l.startTime).toLocaleTimeString([], {hour: '2-digit', minute:'2-digit'})} ~ ${new
Date(l.endTime).toLocaleTimeString([], {hour: '2-digit', minute:'2-digit'})}</span>
    </div>
    <div class="flex justify-between items-center">
      <span class="text-sm font-bold" style="color: ${subjectColors[l.subject] ||
'#e2e8f0'}">${l.subject}</span>
      <span class="font-mono text-gray-700 text-sm
font-bold">${Math.floor(l.duration / 60)}分</span>
    </div>
  </div>`).join("");

document.getElementById('points-display').textContent =
localStorage.getItem('localPoints') || 0;
}
</script>
</body>
</html>

```